

WHY BUILD A FRONTEND FRAMEWORK

Created: 2025-11-16 Sun 21:53

GOALS OF THE LECTURE

GOALS OF THE LECTURE

Learn

GOALS OF THE LECTURE

Learn

tackle the difficulties and explore new ways of interacting with web pages

GOALS OF THE LECTURE

Learn

tackle the difficulties and explore new ways of
interacting with web pages

Concepts

GOALS OF THE LECTURE

Learn

tackle the difficulties and explore new ways of interacting with web pages

Concepts

Learn the concepts of a frontend framework

GOALS OF THE LECTURE

Learn

tackle the difficulties and explore new ways of interacting with web pages

Concepts

Learn the concepts of a frontend framework

Interaction

GOALS OF THE LECTURE

Learn

tackle the difficulties and explore new ways of interacting with web pages

Concepts

Learn the concepts of a frontend framework

Interaction

FEF can be complex but it is just working with the tools provided by the browsers (DOM, CSSOM, JS interpreter).

FRAMEWORK VS. LIBRARY

FRAMEWORK VS. LIBRARY

- When you use a *library*, you import its code and call its functions.

FRAMEWORK VS. LIBRARY

- When you use a *library*, you import its code and call its functions.
- When you use a *framework*, you write code that the framework executes.

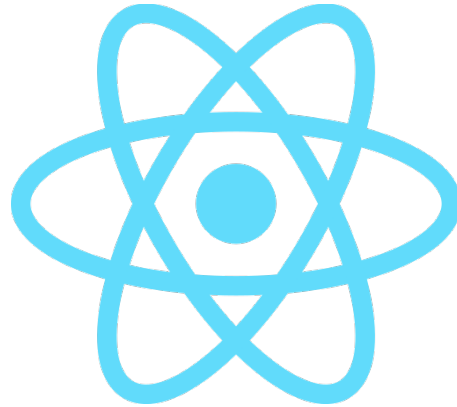
FRAMEWORK VS. LIBRARY

- When you use a *library*, you import its code and call its functions.
- When you use a *framework*, you write code that the framework executes.

The framework is in charge of running the application, and it executes your code when an appropriate trigger happens. Conversely, you call the library's functions when you need them.



Angular is an example of a frontend framework



React claims to be a library that you can use to build user interfaces (UIs).

WHAT WE WILL FOCUS ON

WHAT WE WILL FOCUS ON

- **Virtual DOM** - lightweight DOM representation.

WHAT WE WILL FOCUS ON

- **Virtual DOM** - lightweight DOM representation.
- **reconciliation algorithm** - selects the smallest set of changes required to update the browser's DOM.

WHAT WE WILL FOCUS ON

- **Virtual DOM** - lightweight DOM representation.
- **reconciliation algorithm** - selects the smallest set of changes required to update the browser's DOM.
- **SPA** - modify the URL in the browser's address bar without reloading the page. Explore the browser's [history API](#).

WHAT WE WILL FOCUS ON

- **Virtual DOM** - lightweight DOM representation.
- **reconciliation algorithm** - selects the smallest set of changes required to update the browser's DOM.
- **SPA** - modify the URL in the browser's address bar without reloading the page. Explore the browser's [history API](#).
- **Other building blocks** - Learn about the other building blocks of a FEF.

WHAT WE WILL FOCUS ON

- **Virtual DOM** - lightweight DOM representation.
- **reconciliation algorithm** - selects the smallest set of changes required to update the browser's DOM.
- **SPA** - modify the URL in the browser's address bar without reloading the page. Explore the browser's [history API](#).
- **Other building blocks** - Learn about the other building blocks of a FEF.
- **Combine all of the things together!**

OUR FRAMEWORK GOAL

OUR FRAMEWORK GOAL

- We won't build the next [Vue](#) or React

OUR FRAMEWORK GOAL

- We won't build the next [Vue](#) or React
- Teach how these frameworks work in general so that what they do isn't magic.

- Borrow ideas from a few existing frameworks:
 - [Vue.js](#),
 - [Mithril](#),
 - [Svelte](#),
 - [React](#),
 - [Preact](#),
 - [Angular](#),
 - [HyperApp](#)

VIRTUAL DOM ABSTRACTION

Virtual DOM it's a lightweight representation of the DOM that's used to calculate the smallest set of changes required to update the browser's DOM.

VIRTUAL DOM ABSTRACTION

Virtual DOM it's a lightweight representation of the DOM that's used to calculate the smallest set of changes required to update the browser's DOM.

- Not all frameworks use the virtual DOM abstraction

VIRTUAL DOM ABSTRACTION

Virtual DOM it's a lightweight representation of the DOM that's used to calculate the smallest set of changes required to update the browser's DOM.

- Not all frameworks use the virtual DOM abstraction
- Svelte in particular considers it to be **pure overhead**.

VIRTUAL DOM ABSTRACTION

Virtual DOM it's a lightweight representation of the DOM that's used to calculate the smallest set of changes required to update the browser's DOM.

- Not all frameworks use the virtual DOM abstraction
- Svelte in particular considers it to be **pure overhead**.
- We will use the virtual DOM in our framework

VIRTUAL DOM (EXAMPLE)

The following HTML segment

```
<div class="name">
  <label for="name-input">Name</label>
  <input type="text" id="name-input" />
  <button>Save</button>
</div>
```

- would have a virtual DOM representation

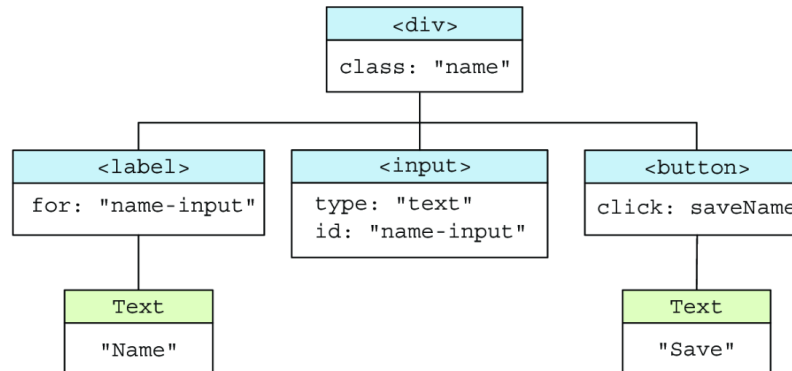


Figure 1: VDOM representation of the HTML markup

VIRTUAL DOM (EXAMPLE PT.2)

- The `saveName ( )` event handler in the figure doesn't appear in the HTML markup.
- Event handlers usually are added programmatically.

RECONCILIATION ALGORITHM

The reconciliation algorithm is the process that decides what changes need to be made to the browser's DOM to reflect the changes in the virtual DOM

COMPONENT BASED ARCHITECTURE

A component-based architecture in which each component does the following:

- Holds its own state
- Manages its own lifecycle
- Re-renders itself and its children when their states change

ADVANCED FEATURES (OPTIONAL)

ADVANCED FEATURES (OPTIONAL)

- An SPA router that updates the URL in the browser's address bar without reloading the page

ADVANCED FEATURES (OPTIONAL)

- An SPA router that updates the URL in the browser's address bar without reloading the page
- Slots for rendering content inside a component

ADVANCED FEATURES (OPTIONAL)

- An SPA router that updates the URL in the browser's address bar without reloading the page
- Slots for rendering content inside a component
- HTML templates that are compiled into JavaScript render functions

ADVANCED FEATURES (OPTIONAL)

- An SPA router that updates the URL in the browser's address bar without reloading the page
- Slots for rendering content inside a component
- HTML templates that are compiled into JavaScript render functions
- Server-side rendering

ADVANCED FEATURES (OPTIONAL)

- An SPA router that updates the URL in the browser's address bar without reloading the page
- Slots for rendering content inside a component
- HTML templates that are compiled into JavaScript render functions
- Server-side rendering
- A browser extension that debugs the framework

ARCHITECTURE OF THE FRAMEWORK

IMPLEMENTATION PLAN

IMPLEMENTATION PLAN

- To build a framework we can use an application that covers the features that we need to implement!

IMPLEMENTATION PLAN

- To build a framework we can use an application that covers the features that we need to implement!
- We will use a simple TODO application to build the framework.

FEATURE LIST

Will implement the following features

1. Implement Virutal DOM
2. Mounting and unmounting
3. State management
4. VDOM reconciliation algorithm

IMPROVING THE FRAMEWORK

1. Stateful components
2. Subcomponents
3. Lifecycle hooks
4. Scheduler
5. Testing

HOW A FRONTEND FRAMEWORK WORKS

Developers Perspective

STARTING POINT

The developer starts by using the framework's command-line interface (CLI) tool by installing the dependencies and configuring the project manually.

USAGE OF NODE.JS

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

Node.js can be optional

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

Node.js can be optional

Developers rely on it as it has a lot of benefits:

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

Node.js can be optional

Developers rely on it as it has a lot of benefits:

- run scripts, and bundle our

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

Node.js can be optional

Developers rely on it as it has a lot of benefits:

- run scripts, and bundle our
- compile and bundle code

USAGE OF NODE.JS

- a *frontend project* is usually a regular Node JS project
 - it has `package.json` file
 - has the `node_modules` directory

Node.js can be optional

Developers rely on it as it has a lot of benefits:

- run scripts, and bundle our
- compile and bundle code
- manage dependencies

COMPONENTS

COMPONENTS

Define part of the application's view and how the user interacts with it. Usually written in HTML, CSS, and JavaScript code. Usually written in a single file (*single-file component - SFC*),

TEMPLATES

TEMPLATES

template is a code that will be compiled into JavaScript render functions.

will enable us to convert the template from HTML

```
<div class="container">  
<h1>Look, Ma!</h1>  
<p>I'm building a framework!</p>  
</div>
```

to JavaScript function for rendering:

```
function render() {  
return h('div', { class: 'container' }, [  
  h('h1', {}, ['Look, Ma!']),  
  h('p', {}, ["I'm building a framework!"])  
)
```

The templates are done in one of two ways

JSX (JavaScript XML)

- It is an extension of JavaScript
- Used by React & Preact.

```
const App = () => {  
  const hasDiscount = 1;  
  return (  
    <div>  
      {hasDiscount && <p>You get a discount!</p>}  
    </div>  
  );  
}
```

Directives

- Extend *HTML* (based) code with custom directives
- Used by Vue, Svelte, Angular etc.

```
{#if hasDiscount}  
<p>  
Discount!  
</p>  
{/if}
```

BUNDLING

Transforming the code into fewer files than originally written so that the browser can load the application by making fewer requests to the server.

The files can also be *minified* – made smaller

- removing whitespace
- removing comments
- giving variables shorter names

BUILDING

BUILDING

- Before we can deploy a frontend application to production, we need to build it.

BUILDING

- Before we can deploy a frontend application to production, we need to build it.
- Done by the framework using a CLI tool

BUILDING

- Before we can deploy a frontend application to production, we need to build it.
- Done by the framework using a CLI tool
- Usually an npm script: `npm run build`

BUILDING STEPS:

BUILDING STEPS:

- Transform the template for each component into JavaScript code.

BUILDING STEPS:

- Transform the template for each component into JavaScript code.
- Bundle all components' code into a single JavaScript file, `app.bundle.js`.
 - enable *lazy-loading* if necessary.

BUILDING STEPS:

- Transform the template for each component into JavaScript code.
- Bundle all components' code into a single JavaScript file, `app.bundle.js`.
 - enable *lazy-loading* if necessary.
- Bundle used third-party code used into `vendors.bundle.js`.

BUILDING STEPS:

- Transform the template for each component into JavaScript code.
- Bundle all components' code into a single JavaScript file, `app.bundle.js`.
 - enable *lazy-loading* if necessary.
- Bundle used third-party code used into `vendors.bundle.js`.
- Extract & bundle CSS code from the components into `bundle.css`.

BUILDING STEPS:

- Transform the template for each component into JavaScript code.
- Bundle all components' code into a single JavaScript file, `app.bundle.js`.
 - enable *lazy-loading* if necessary.
- Bundle used third-party code used into `vendors.bundle.js`.
- Extract & bundle CSS code from the components into `bundle.css`.
- Generate `index.html` for serving to the user

BUILDING STEPS:

- Transform the template for each component into JavaScript code.
- Bundle all components' code into a single JavaScript file, `app.bundle.js`.
 - enable *lazy-loading* if necessary.
- Bundle used third-party code used into `vendors.bundle.js`.
- Extract & bundle CSS code from the components into `bundle.css`.
- Generate `index.html` for serving to the user
- Preprocess, optimize and copy the static assets (images, fonts, and audio clips)

A typical build process results in four files (or more, in the case of larger apps):

A typical build process results in four files (or more, in the case of larger apps):

- `app.bundle.js` - with the application's code.

A typical build process results in four files (or more, in the case of larger apps):

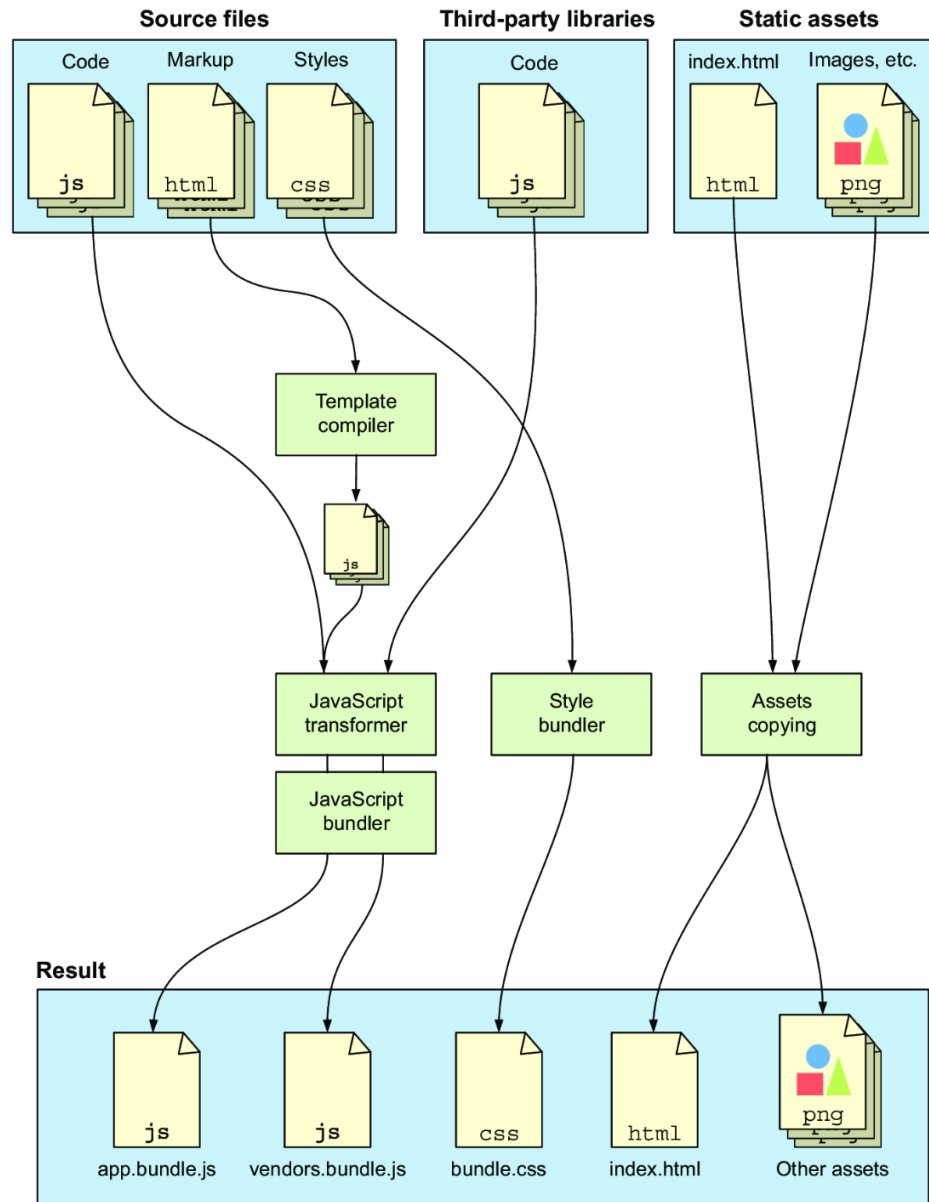
- `app.bundle.js` - with the application's code.
- `vendors.bundle.js` - with the third-party code

A typical build process results in four files (or more, in the case of larger apps):

- `app.bundle.js` - with the application's code.
- `vendors.bundle.js` - with the third-party code
- `bundle.css` - with the application's CSS

A typical build process results in four files (or more, in the case of larger apps):

- `app.bundle.js` - with the application's code.
- `vendors.bundle.js` - with the third-party code
- `bundle.css` - with the application's CSS
- `index.html` - the HTML file that will be served to the user



DEPLOYMENT

- The generated files are uploaded to a server, and the application is ready to be served to the user.
- When a user requests the website, the HTML, JS, and CSS files are statically served.

Let's see what happens in the browser after these files are loaded. The flow is slightly different depending on whether the application server-side rendered (SSR) or statically served as an SPA. Let's start with the slightly simpler latter case.

HOW A FRONTEND FRAMEWORK WORKS

The browser side of an SPA

In an SPA, the server responds with a mostly empty HTML file that's used to load the application's JavaScript and CSS files.

In an SPA, the server responds with a mostly empty HTML file that's used to load the application's JavaScript and CSS files.

- Use of the Document API to create and update the application's view.

In an SPA, the server responds with a mostly empty HTML file that's used to load the application's JavaScript and CSS files.

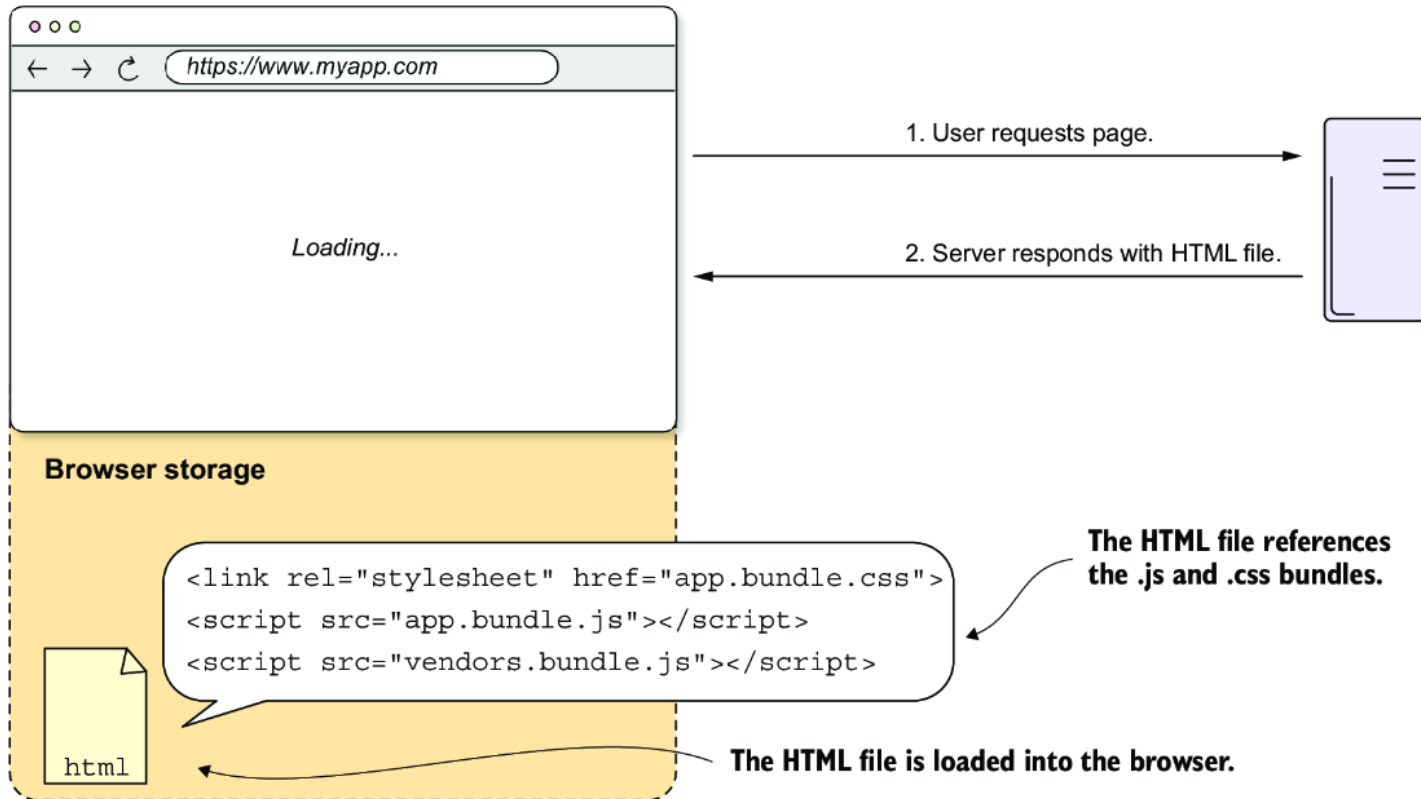
- Use of the Document API to create and update the application's view.
- A **router** makes sure that the entire application isn't reloaded when the user navigates to a different URL

In an SPA, the server responds with a mostly empty HTML file that's used to load the application's JavaScript and CSS files.

- Use of the Document API to create and update the application's view.
- A **router** makes sure that the entire application isn't reloaded when the user navigates to a different URL
 - update the view and the URL to show the new content and enable navigation

STEP 1: LOADING THE HTML FILE

When the user navigates to the application by writing its URL, the browser requests the page's HTML file (1), which is returned by the server.



The browser loads the HTML file and parses it. This HTML, which is mostly empty, is used to load the JavaScript and CSS bundles declared in the `<script>` and `<link>` tags. These bundles are the application

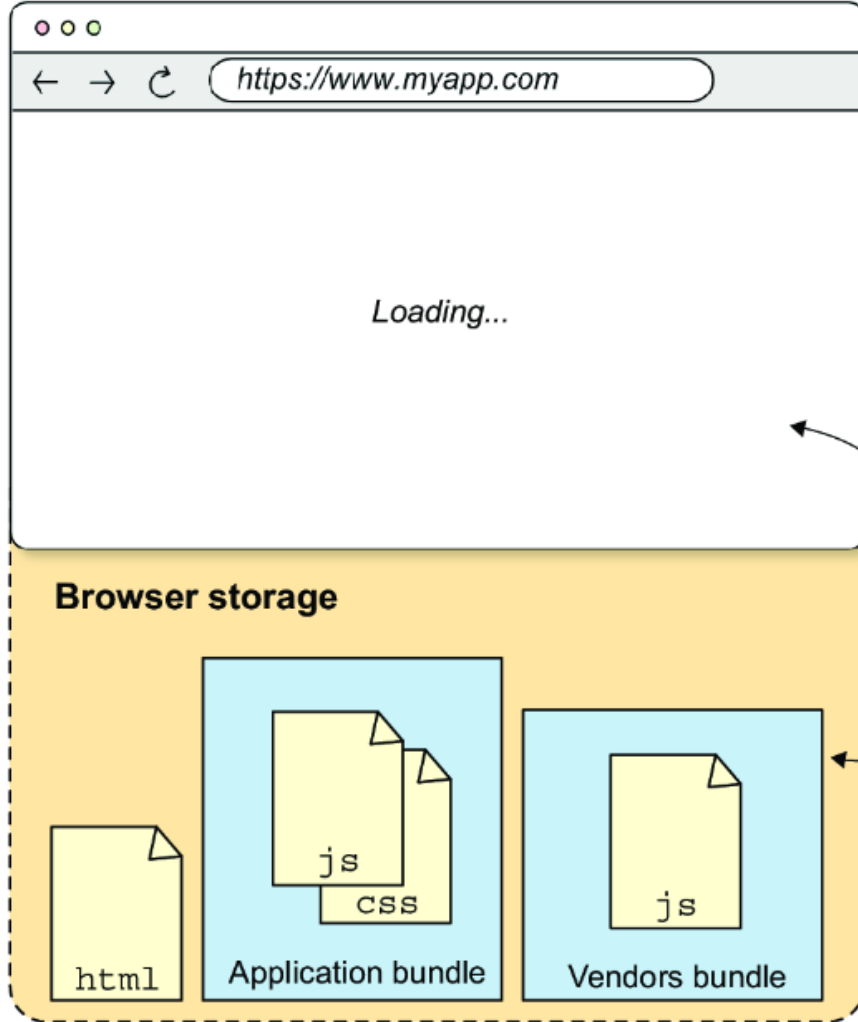
STEP 2: LOADING THE JAVASCRIPT AND CSS FILES

STEP 2: LOADING THE JAVASCRIPT AND CSS FILES

- load the JavaScript and CSS files referenced in the HTML

STEP 2: LOADING THE JAVASCRIPT AND CSS FILES

- load the JavaScript and CSS files referenced in the HTML
- parse the JavaScript code referenced in the HTML.



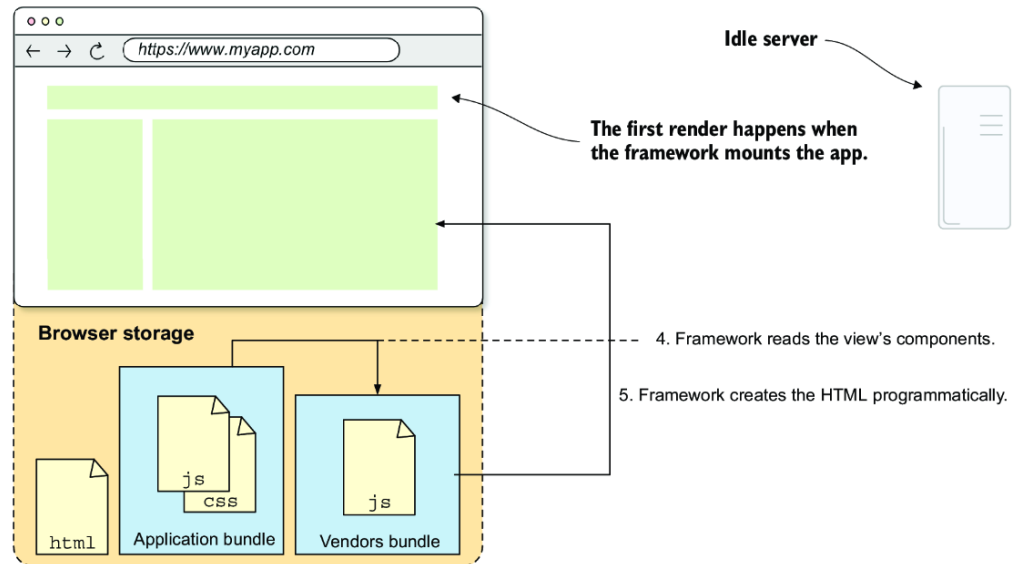
3. Browser loads .js and .css files.

Blank browser. Nothing has been rendered yet (except the mostly empty HTML file).

Application and vendor bundles are loaded into the browser.

STEP 3: CREATING THE APPLICATION'S VIEW (MOUNTING THE APP)

Find the components that need to be rendered and do the initial rendering using the Document API.



STEP 4: HANDLING USER INTERACTIONS

STEP 4: HANDLING USER INTERACTIONS

- When user interacts with the application, the framework handles the event and updates the view accordingly.

STEP 4: HANDLING USER INTERACTIONS

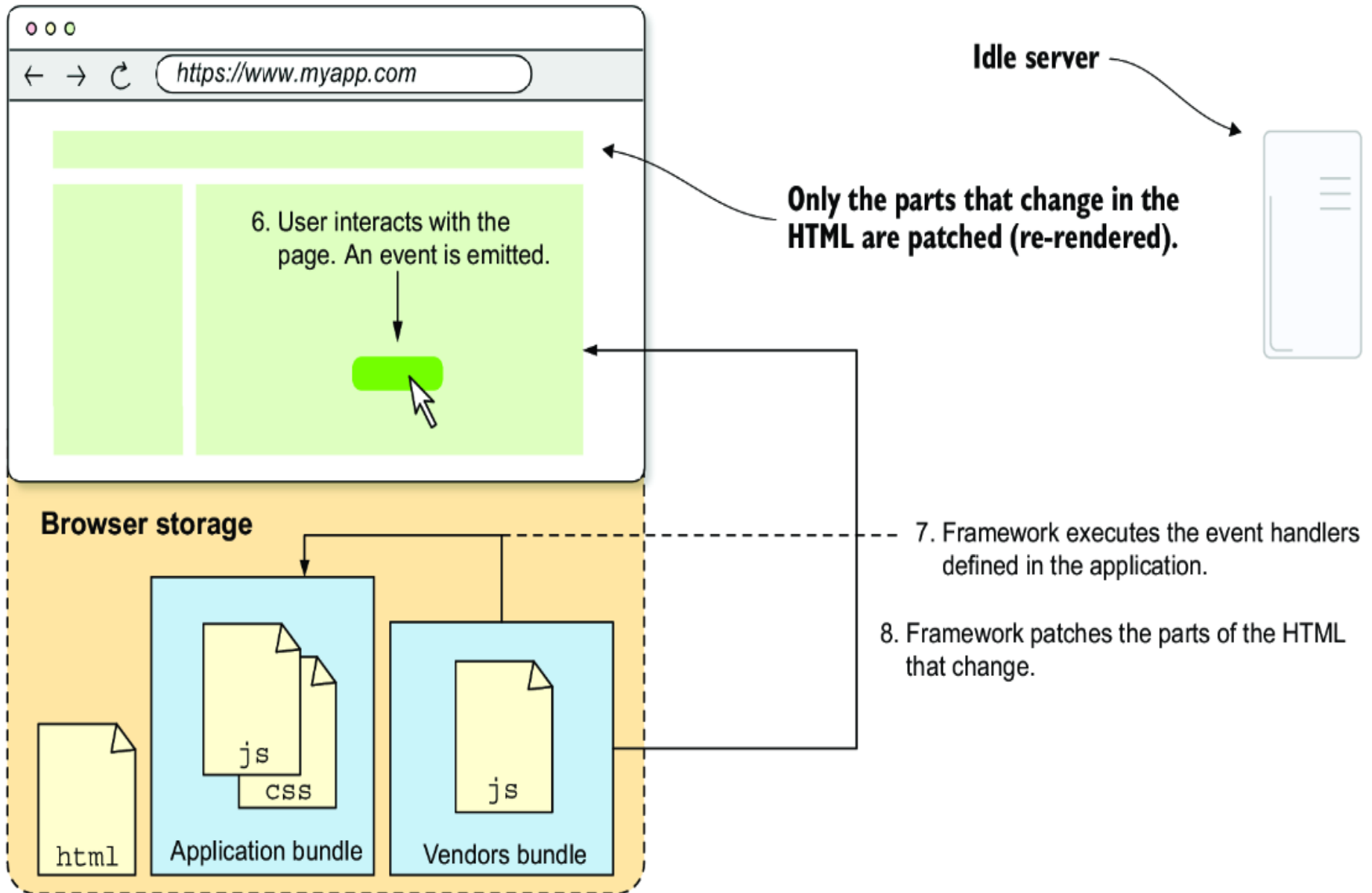
- When user interacts with the application, the framework handles the event and updates the view accordingly.
- The framework is responsible for updating only the parts of the HTML that need to be updated.

STEP 4: HANDLING USER INTERACTIONS

- When user interacts with the application, the framework handles the event and updates the view accordingly.
- The framework is responsible for updating only the parts of the HTML that need to be updated.
 - This process is called **patching the DOM**

STEP 4: HANDLING USER INTERACTIONS

- When user interacts with the application, the framework handles the event and updates the view accordingly.
- The framework is responsible for updating only the parts of the HTML that need to be updated.
 - This process is called **patching the DOM**
 - The number of updates should be minimal (expensive process).



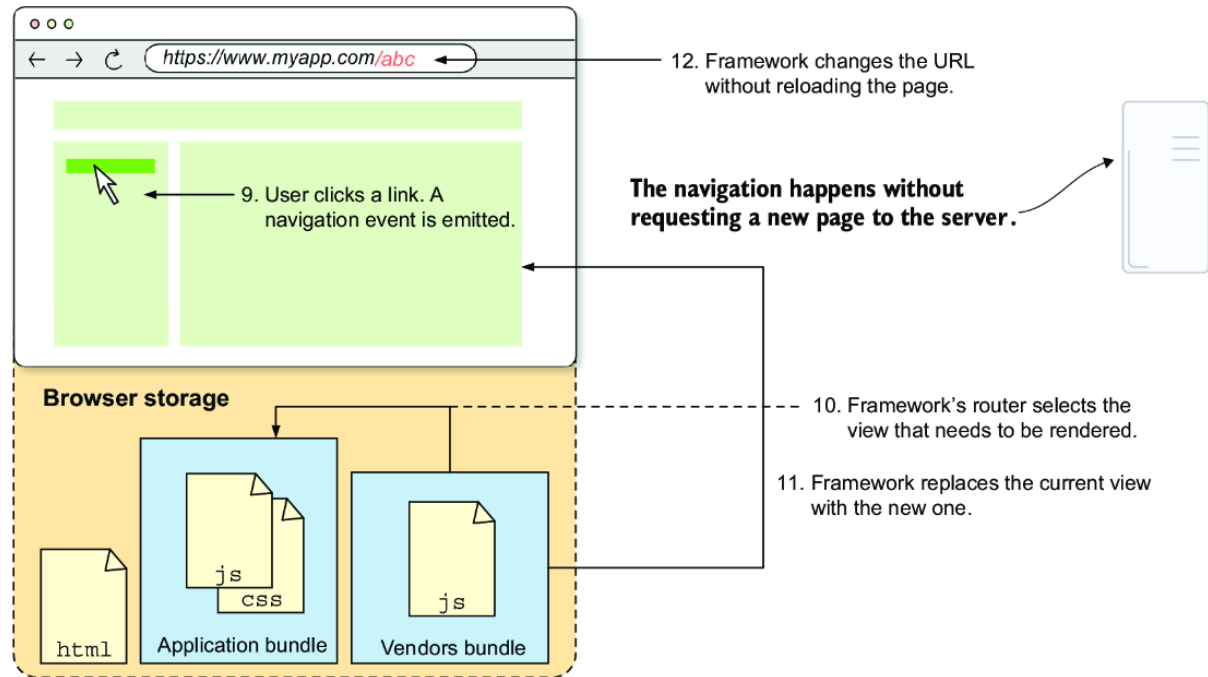
Definition: A single change that the framework makes in the DOM is called a patch. The process of updating the view to reflect the changes in the application's state is called patching the DOM.

How some frameworks update the view:

- **Svelte** deduces possible changes at compilation time (very performant).
- **Angular** runs a change detection routine, based on **zones**.
- Other frameworks (e.g. Vue, React, Preact,) use a virtual DOM representation.

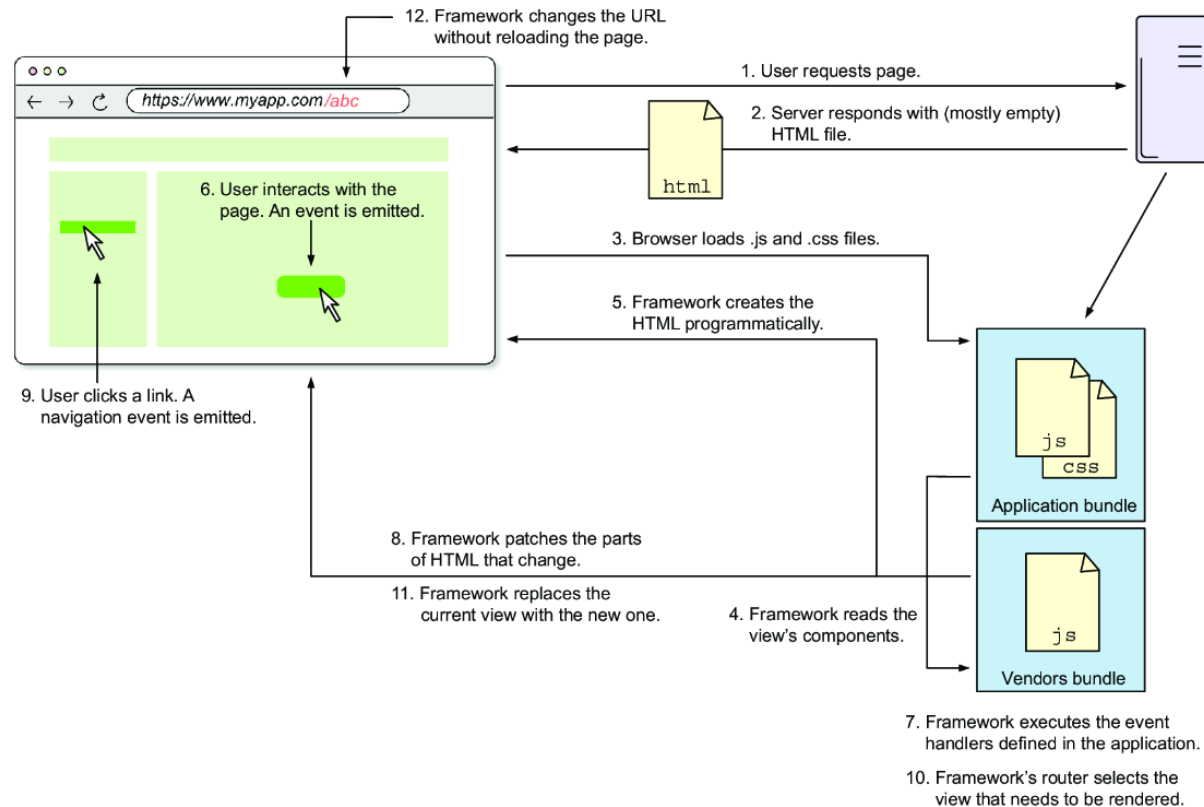
STEP 5: NAVIGATING AMONG ROUTES

- When the user clicks a link, the framework's router prevents the default behavior of reloading the page;
- Instead it renders the component that's configured for the new route.
- The router is also in charge of changing the URL to reflect the new route



- A SPA works with a single HTML file in which the HTML markup code is updated programmatically by the framework,
- new HTML pages aren't requested to the server

THE COMPLETE FLOW OF AN SPA



HOW A FRONTEND FRAMEWORK WORKS

The browser and server sides of an **Server-Side Rendering (SSR)** application

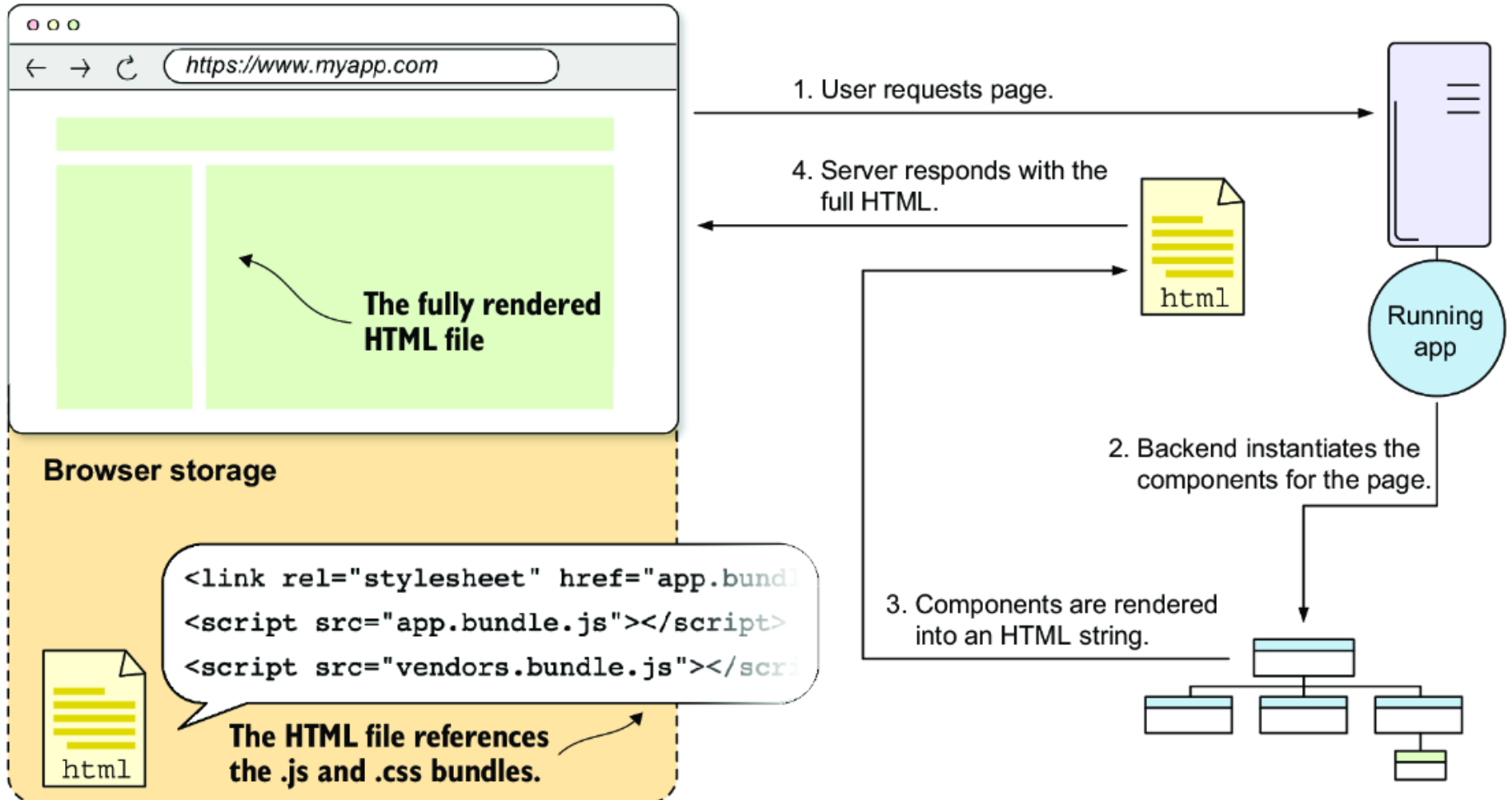
SSRs are web applications that render the HTML markup on the server and send it to the browser.

- backend is required to handle requests and render the HTML pages.

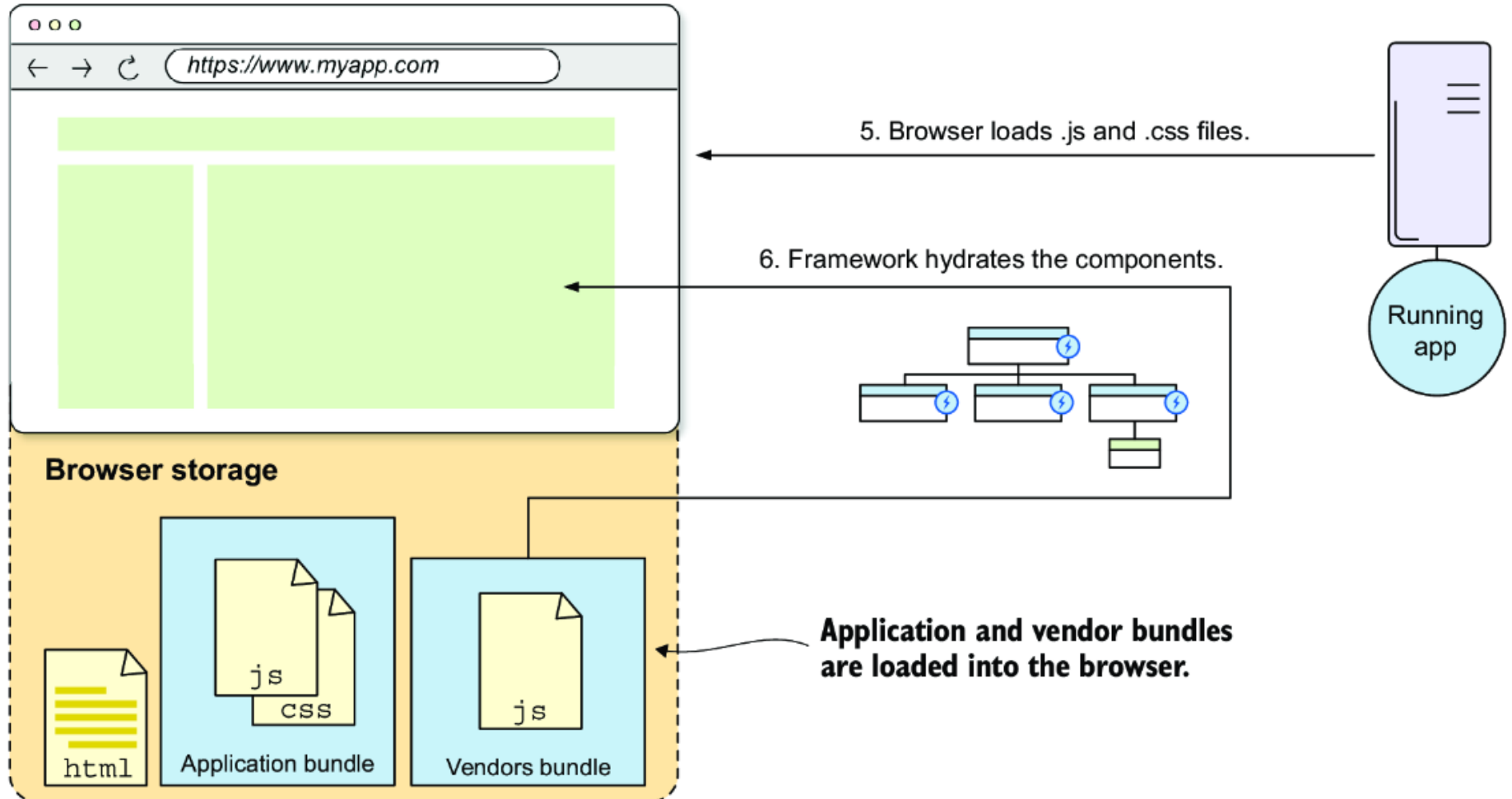
- backend is required to handle requests and render the HTML pages.
- frontend code is responsible for handling user interactions
 - updating the view to reflect the changes in the application's state

- backend is required to handle requests and render the HTML pages.
- frontend code is responsible for handling user interactions
 - updating the view to reflect the changes in the application's state
- when the user navigates to a different route, the browser requests a new HTML page from the server instead of updating the HTML markup programmatically.

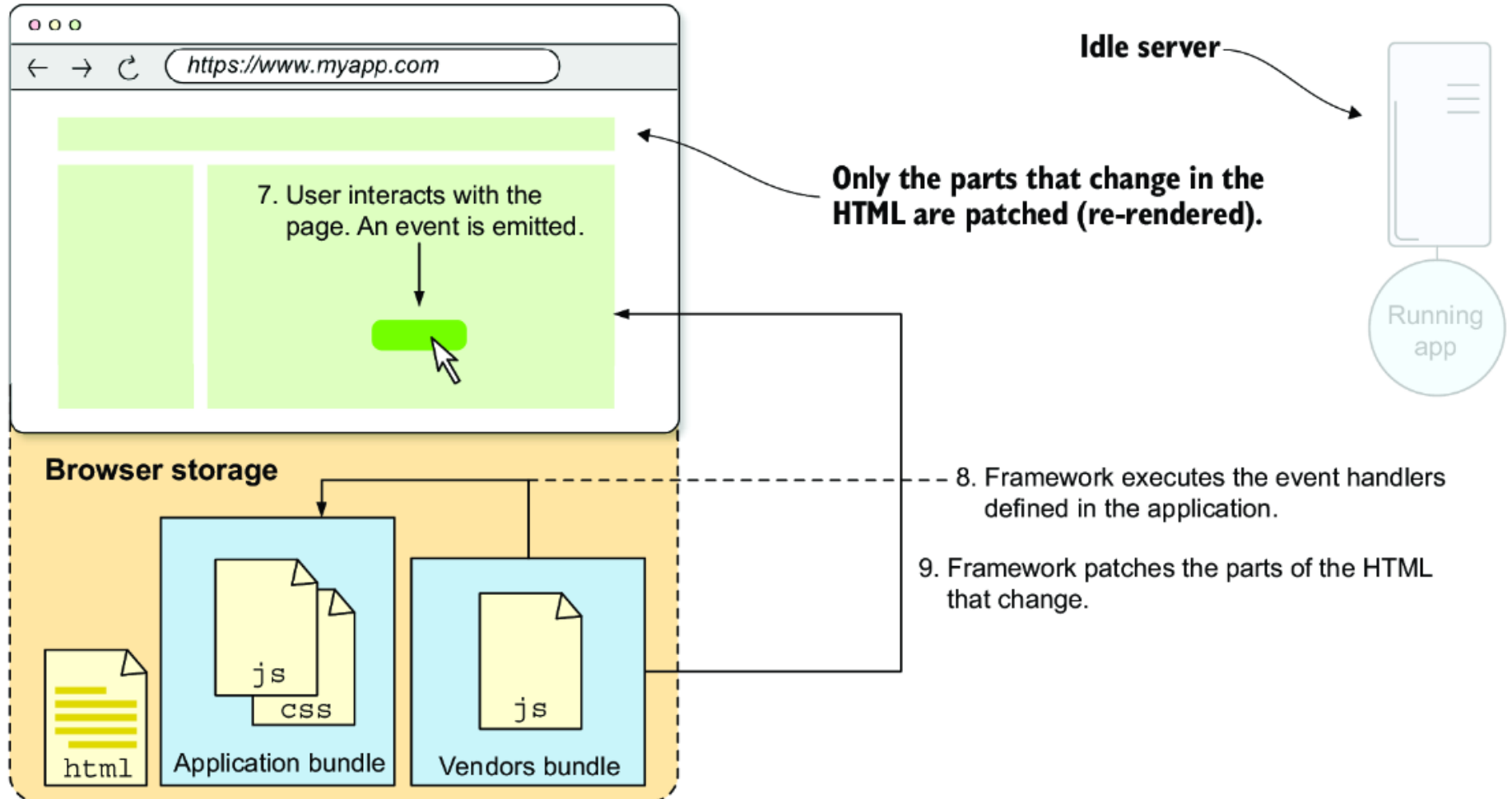
STEP 1: LOADING AN HTML PAGE



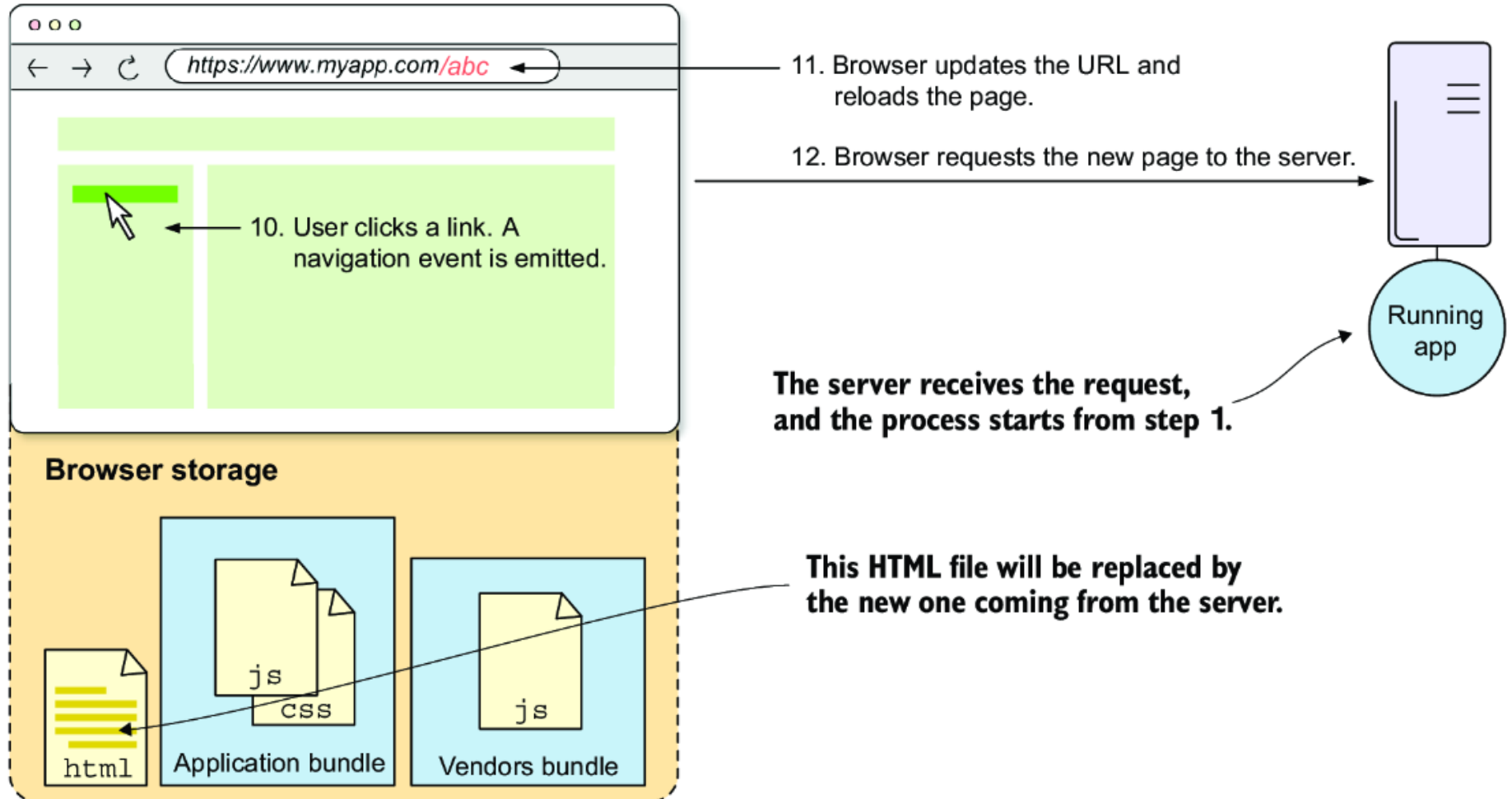
STEP 2: HYDRATING THE HTML PAGE



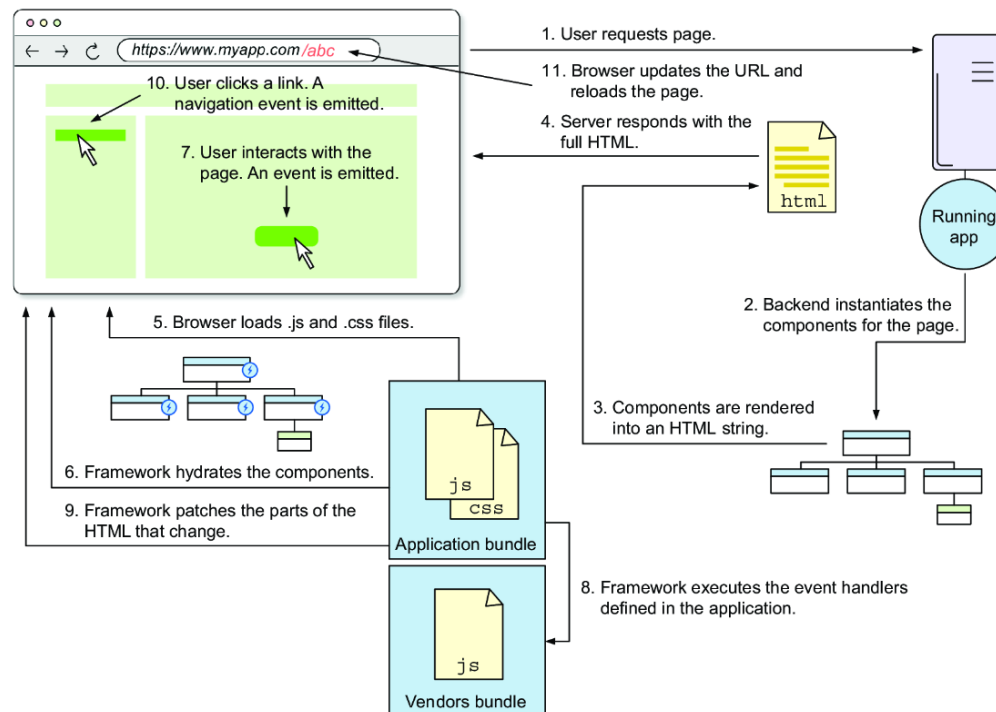
STEP 3: HANDLING USER INTERACTIONS



STEP 4: NAVIGATING BETWEEN ROUTES



THE COMPLETE FLOW OF AN SSR APPLICATION



SUMMARY

- Building a frontend framework from scratch is a great way to learn how frameworks operate.
- Frontend frameworks bundle the application's code into a single JavaScript file, the third-party dependencies into another file, and the CSS styles into yet another file. If the application is large, the framework might split the application's code into multiple bundles that are loaded lazily (as needed).

The Document API allows HTML elements to be created programmatically, using JavaScript. Frontend frameworks use the Document API to create the application's view.

SPAs consist of a single HTML file that's loaded by the browser and updated by the framework to reflect the application's state. When the route changes, the framework changes the view; the browser doesn't reload the page.

The hydration process connects the existing HTML markup, rendered in the server, to the component's state and event listeners.